Unit 1

1.introduction of python.

Python is a widely used general-purpose, high level programming language. It was created by Guido van Rossum in 1991 and further developed by the Python Software Foundation. It was designed with an emphasis on code readability, and its syntax allows programmers to express their concepts in fewer lines of code.

Python is a programming language that lets you work quickly and integrate systems more efficiently.

Python is a general-purpose, dynamic, high-level, and interpreted programming language. It supports Object Oriented programming approach to develop applications. It is simple and easy to learn and provides lots of high-level data structures.

Python is an easy-to-learn yet powerful and versatile scripting language, which makes it attractive for Application Development.

Instead of Semicolon as used in other languages, Python ends its statements with a NewLine character.

Python is a case-sensitive language, which means that uppercase and lowercase letters are treated differently. For example, 'name' and 'Name' are two different variables in Python. B

Python laid its foundation in the late 1980s.

The implementation of Python was started in December 1989 by Guido Van Rossum at CWI in Netherland.

In February 1991, Guido Van Rossum published the code (labeled version 0.9.0) to alt.sources.

In 1994, Python 1.0 was released with new features like lambda, map, filter, and reduce.

Python 2.0 added new features such as list comprehensions, garbage collection systems.

Versions of python.

Python 1.0	January 1994
Python 1.5	December 31, 1997
Python 1.6	September 5, 2000
Python 2.0	October 16, 2000
Python 2.1	April 17, 2001
Python 2.2	December 21, 2001
Python 2.3	July 29, 2003
Python 2.4	November 30, 2004
Python 2.5	September 19, 2006
Python 2.6	October 1, 2008

Python 2.7	July 3, 2010
Python 3.0	December 3, 2008
Python 3.1	June 27, 2009
Python 3.2	February 20, 2011
Python 3.3	September 29, 2012
Python 3.4	March 16, 2014
Python 3.5	September 13, 2015
Python 3.6	December 23, 2016
Python 3.7	June 27, 2018
Python 3.8	October 14, 2019

Difference between python and Java.

Python

Python is a high-level, interpreted programming language. It was invented back in 1991, by Guido Van Rossum. Python is an object-oriented programming language that has large enormous library support making the implementation of various programs and algorithms easy. Its language constructs and object-oriented approach aim to help programmers to write clear, logical code for various projects. The biggest strength of Python is its huge collection of standard libraries which can be used for the following:

Machine Learning

GUI Applications (like Kivy, Tkinter, PyQt, etc.)

Web frameworks like Django (used by YouTube, Instagram, and Dropbox)

Image processing (like OpenCV, Pillow)

Web scraping (like Scrapy, BeautifulSoup, and Selenium)

Test frameworks

Multimedia

Scientific computing

Text processing and many more.

Java

Java is a high-level, object-oriented programming language that was originally developed by James Gosling at Sun Microsystems in 1995. Java has a syntax similar to C and C++ but with low-level difficulties. Java is platform-independent (WORA – Write Once Run Anywhere) meaning compiled java code can run on different platforms without recompilation. Java can be used to develop the following applications:

Desktop GUI Applications

Mobile Applications

Artificial intelligence

Web applications

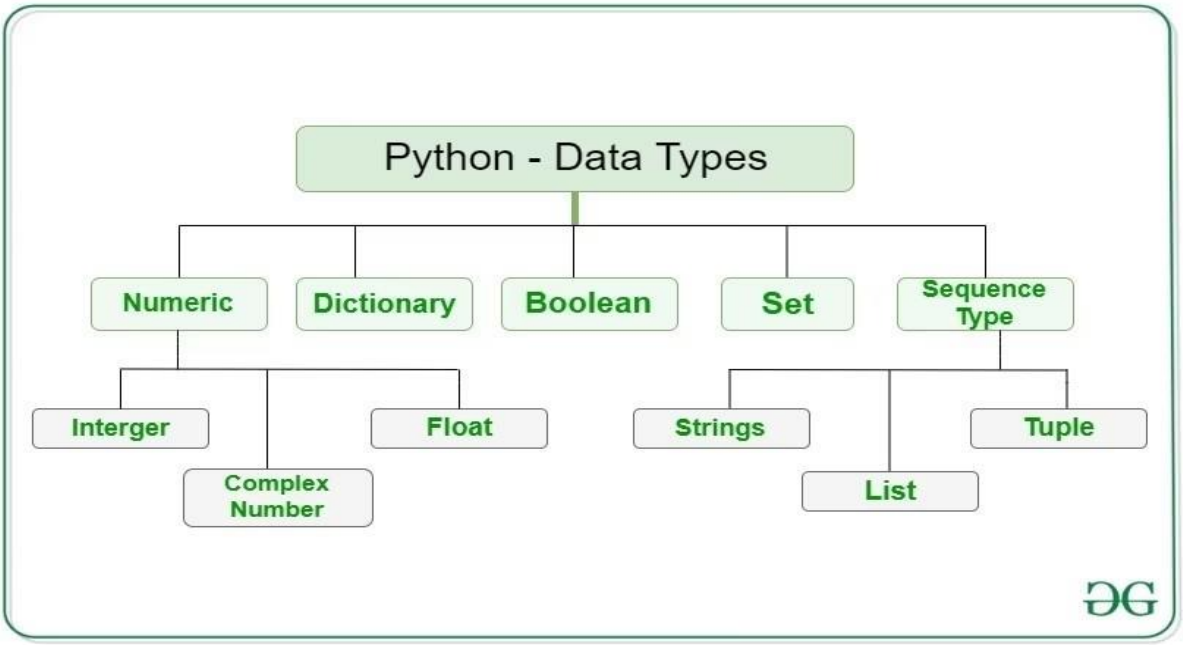
Big Data technology

Gaming applications

Business applications

Python Data Types

Data types are the classification or categorization of data items. It represents the kind of value that tells what operations can be performed on a particular data. Since everything is an object in Python programming, data types are actually classes and variables are instances (object) of these classes. The following are the standard or built-in data types in Python:



Type() function

To define the values of various data types and check their data types we use the type() function. Consider the following examples.

This code assigns variable ‘x’ different values of various data types in Python. It covers string, integer, float, complex, list, tuple, range, dictionary, set, frozenset, boolean, bytes, bytearray, memoryview, and the special value ‘None’ successively. Each assignment replaces the previous value, making ‘x’ take on the data type and value of the most recent assignment.

Numeric data type.

The numeric data type in Python represents the data that has a numeric value. A numeric value can be an integer, a floating number, or even a complex number. These values are defined as Python int, Python float, and Python complex classes in Python.

Integers – This value is represented by int class. It contains positive or negative whole numbers (without fractions or decimals). In Python, there is no limit to how long an integer value can be.

Float – This value is represented by the float class. It is a real number with a floating-point representation. It is specified by a decimal point. Optionally, the character e or E followed by a positive or negative integer may be appended to specify scientific notation.

Complex Numbers – Complex number is represented by a complex class. It is specified as (real part) + (imaginary part)For example – 2+3j

A = 5

Print("Type of a: ", type(a))

B = 5.0

Print("\nType of b: ", type(b))

C = 2 + 4j

Print("\nType of c: ", typec)

Output:

Type of a: <class 'int'>

Type of b: <class 'float'>

Type of c: <class 'complex'>

String Data type

Strings in Python are arrays of bytes representing Unicode characters. A string is a collection of one or more characters put in a single quote, double-quote, or triple-quote. In python there is no character data type, a character is a string of length one. It is represented by str class.

Print("hello world!")

Print("hello world!")

Print("hello world!")

O/P:-

Hello world!

Hello world!

Hello world!

Creating String

Strings in Python can be created using single quotes or double quotes or even triple quotes.

Accessing elements of String

In Python, individual characters of a String can be accessed by using the method of Indexing. Negative Indexing allows negative address references to access characters from the back of the String, e.g. -1 refers to the last character, -2 refers to the second last character, and so on.

List Data Type

Lists are just like arrays, declared in other languages which is an ordered collection of data. It is very flexible as the items in a list do not need to be of the same type.

List1=[“hello”,”friends”,”good”,”morning”]

Print(list1[2])

Print(list1[-3])

Print(list1[1:])

Print(list1[1:3])

O/P:-

Good

Friends

[‘friends’,’good’,’morning’]

[‘friends’,’good’]

Creating List in Python

Lists in Python can be created by just placing the sequence inside the square brackets[].

Python Access List Items

In order to access the list items refer to the index number. Use the index operator [ ] to access an item in a list. In Python, negative sequence indexes represent positions from the end of the array.

Tuple Data Type

Just like a list, a tuple is also an ordered collection of Python objects. The only difference between a tuple and a list is that tuples are immutable i.e. tuples cannot be modified after it is created. It is represented by a tuple class.

Creating a Tuple in Python

In Python, tuples are created by placing a sequence of values separated by a ‘comma’ with or without the use of parentheses for grouping the data sequence. Tuples can contain any number of elements and of any datatype (like strings, integers, lists, etc.).

Access Tuple Items

In order to access the tuple items refer to the index number. Use the index operator [ ] to access an item in a tuple. The index must be an integer. Nested tuples are accessed using nested indexing.

The code reates a tuple named 'tuple1' with five elements: 1, 2, 3, 4, and 5. Then it prints the first, last, and third last elements of the tuple using indexing.

Example:-

```
 Tuple1=("admin","Justin","jack")

 For I in range (len (tuple1)):

 Print(tuple[i])
```

O/P:-

Admin

Just

Jackin

Set Data Type in Python

In Python, a Set is an unordered collection of data types that is iterable, mutable and has no duplicate elements. The order of elements in a set is undefined though it may consist of various elements.

Create a Set in Python

Sets can be created by using the built-in set() function with an iterable object or a sequence by placing the sequence inside curly braces, separated by a 'comma'. The type of elements in a set need not be the same, various mixed-up data type values can also be passed to the set.

Access Set Items

Set items cannot be accessed by referring to an index, since sets are unordered the items has no index. But you can loop through the set items using a for loop, or ask if a specified value is present in a set, by using the in the keyword.

```
Set1 = set()

Set2 = {'James', 2, 3,'Python'}

#Printing Set value

Print(set2)

# Adding element to the set

Set2.add(10)

Print(set2)

  #Removing element from the set
```

Set2.remove(2)

Print(set2)

Output:

{3, ‘Python’, ‘James’, 2}

{‘Python’, ‘James’, 3, 2, 10}

{‘Python’, ‘James’, 3, 10}

Dictionary Data Type in Python

A dictionary in Python is an unordered collection of data values, used to store data values like a map, unlike other Data Types that hold only a single value as an element, a Dictionary holds a key: value pair. Key-value is provided in the dictionary to make it more optimized. Each key-value pair in a Dictionary is separated by a colon : , whereas each key is separated by a ‘comma’.

Create a Dictionary in Python

In Python, a Dictionary can be created by placing a sequence of elements within curly {} braces, separated by ‘comma’. Values in a dictionary can be of any datatype and can be duplicated, whereas keys can’t be repeated and must be immutable. The dictionary can also be created by the built-in function dict(). An empty dictionary can be created by just placing it in curly braces{}.

Accessing Key-value in Dictionary

In order to access the items of a dictionary refer to its key name. Key can be used inside square brackets. There is also a method called get() that will also help in accessing the element from a dictionary.

Example

D = {1:’Jimmy’, 2:’Alex’, 3:’john’, 4:’mike’}

Print (d)

Print(“1st name is “+d[1])

Print(“2nd name is “+ d[4])

Print (d.keys())

Print (d.values())

Output:

1st name is Jimmy

2nd name is mike

{1: ‘Jimmy’, 2: ‘Alex’, 3: ‘john’, 4: ‘mike’}

Dict_keys([1, 2, 3, 4])

Dict_values([‘Jimmy’, ‘Alex’, ‘john’, ‘mike’])

Variables in python.

Python Variable is containers that store values. Python is not “statically typed”. We do not need to declare variables before using them or declare their type. A variable is created the moment we first assign a value to it. A Python variable is a name given to a memory location. It is the basic unit of storage in a program.

Since Python is an infer language that is smart enough to determine the type of a variable, we do not need to specify its type in Python.

Variable names must begin with a letter or an underscore, but they can be a group of both letters and digits.

The name of the variable should be written in lowercase. Both Rahul and rahul are distinct variables.

Rules for Python variables

A Python variable name must start with a letter or the underscore character.

A Python variable name cannot start with a number.

A Python variable name can only contain alpha-numeric characters and underscores (A-z, 0-9, and _).

Variable in Python names are case-sensitive (name, Name, and NAME are three different variables).

The reserved words(keywords) in Python cannot be used to name the variable in Python.

Variables Assignment in Python

Here, we have assigned a number, a floating point number, and a string to a variable such as age, salary, and name.

```
# An integer assignment

Age = 45

# A floating point

Salary = 1456.8

# A string

Name = “John”

Print(age)

Print(salary)

Print(name)
```

Output:

```
45

1456.8

John
```

Declaration and Initialization of Variables

Let's see how to declare the variable and print the variable.

```
# declaring the var

Number = 100

# display

Print( Number)
```

Output:

100

Creating Variables

Python has no command for declaring a variable.

A variable is created the moment you first assign a value to it.

```
X = 5

Y = "John"

Print(x)

Print(y)
```

Casting

If you want to specify the data type of a variable, this can be done with casting.

Example

```
X = str(3)    # x will be '3'

Y = int(3)    # y will be 3

Z = float(3)  # z will be 3.0
```

Operators in python

In Python programming, Operators in general are used to perform operations on values and variables. These are standard symbols used for the purpose of logical and arithmetic operations. In this article, we will look into different types of Python operators.

OPERATORS: These are the special symbols. Eg- + , * , /, etc.

OPERAND: It is the value on which the operator is applied.

1.Arithmetic Operators in Python

Python Arithmetic operators are used to perform basic mathematical operations like addition, subtraction, multiplication, and division.

Operator	Name	Example
+	Addition	x + y
-	Subtraction	x – y
*	Multiplication	x * y
/	Division.	x / y
%	Modulus	x % y
**	Exponentiation	x ** y
//	Floor division	x // y

2.Comparison Operators in Python

In Python Comparison of Relational operators compares the values. It either returns True or False according to the condition.

Operator	Name	Example
==	Equal	x == y
!=	Not equal	x != y
➢	Greater than	x > y
<	Less than	x < y
>=	Greater than or equal to	x >= y
<=	Less than or equal to	x <= y

3.Logical Operators in Python

Python Logical operators perform Logical AND, Logical OR, and Logical NOT operations. It is used to combine conditional statements.

Operator	Description	Syntax
And	Logical AND: True if both the operands are true.	x and y
Or	Logical OR: True if either of the operands is true	x or y
Not	Logical NOT: True if the operand is false	not x

4.Bitwise Operators in Python

Python Bitwise operators act on bits and perform bit-by-bit operations. These are used to operate on binary numbers.

Operator	Description	Syntax
&	Bitwise AND	x & y
|	Bitwise OR	x | y
~	Bitwise NOT	~x

^ Bitwise XOR. x ^ y

>> Bitwise right shift x>>

<< Bitwise left shift x<<

5.Identity Operators in Python

In Python, is and is not are the identity operators both are used to check if two values are located on the same part of the memory. Two variables that are equal do not imply that they are identical.

Is True if the operands are identical

Is not True if the operands are not identical

6.Membership Operators in Python

In Python, in and not in are the membership operators that are used to test whether a value or variable is in a sequence.

In True if value is found in the sequence

Not in True if value is not found in the sequence

Decision control structure

Python if statement

The if statement is the most simple decision-making statement. It is used to decide whether a certain statement or block of statements will be executed or not.

Syntax:

```
If condition:
  # Statements to execute if
  # condition is true
```

Here, the condition after evaluation will be either true or false. If the statement accepts boolean values – if the value is true then it will execute the block of statements below it otherwise not.

```
I = 10
If (I > 15):
   Print("10 is less than 15")
Print("I am Not in if")
```

Output:

```
I am Not in if
```

Python If-Else Statement

The if statement alone tells us that if a condition is true it will execute a block of statements and if the condition is false it won't. But if we want to do something else if the condition is false, we can use the else statement with if statement to execute a block of code when the if condition is false.

Syntax of Python If-Else:

```
If (condition):
   # Executes this block if
   # condition is true
Else:
   # Executes this block if
   # condition is false
```

Nested-If Statement in Python

A nested if is an if statement that is the target of another if statement. Nested if statements mean an if statement inside another if statement. Yes, Python allows us to nest if statements within if statements. i.e., we can place an if statement inside another if statement.

Syntax:

```
If (condition1):
  # Executes when condition1 is true
  If (condition2):
    # Executes when condition2 is true
  # if Block is end here
# if Block is end here
```

Python if-elif-else Ladder

Here, a user can decide among multiple options. The if statements are executed from the top down. As soon as one of the conditions controlling the if is true, the statement associated with that if is executed, and the rest of the ladder is bypassed. If none of the conditions is true, then the final else statement will be executed.

Syntax:

```
If (condition):
   Statement
Elif (condition):
   Statement
.
.
Else:
   Statement
```

Looping statement in python.

While Loop in Python

In Python, a while loop is used to execute a block of statements repeatedly until a given condition is satisfied. And when the condition becomes false, the line immediately after the loop in the program is executed.

While Loop Syntax:

```
While expression:
   Statement(s)
```

All the statements indented by the same number of character spaces after a programming construct are considered to be part of a single block of code. Python uses indentation as its method of grouping statements.

For Loop in Python

For loops are used for sequential traversal. For example: traversing a list or string or array etc. In Python, there is “for in” loop which is similar to for each loop in other languages. Let us learn how to use for in loop for sequential traversals.

For Loop Syntax:

```
For iterator_var in sequence:
   Statements(s)
```

It can be used to iterate over a range and iterators.

The code uses a Python for loop that iterates over the values from 0 to 3 (not including 4), as specified by the range(0, n) construct. It will print the values of ‘I’ in each iteration of the loop.

```
N = 4
For I in range(0, n):
   Print(i)
```

Output

0

1

2

3

The range() function defaults to increment the sequence by 1, however it is possible to specify the increment value by adding a third parameter: range(2, 30, 3):

```
For x in range(2, 30, 3):
  Print(x)
```

Nested Loops

Python programming language allows to use one loop inside another loop. Following section shows few examples to illustrate the concept.

Nested Loops Syntax:

```
For iterator_var in sequence:
  For iterator_var in sequence:
    Statements(s)
  Statements(s)
```

Loop Control Statements

Loop control statements change execution from their normal sequence. When execution leaves a scope, all automatic objects that were created in that scope are destroyed. Python supports the following control statements.

Continue Statement

The continue statement in Python returns the control to the beginning of the loop.

```
Fruits = [“apple”, “banana”, “cherry”]
For x in fruits:
 If x == “banana”:
  Continue
 Print(x)
```

Break Statement

The break statement in Python brings control out of the loop.

```
Fruits = [“apple”, “banana”, “cherry”]
For x in fruits:
 Print(x)
 If x == “banana”:
  Break
```

Pass Statement

We use pass statement in Python to write empty loops. Pass is also used for empty control statements, functions and classes

```
For x in [0, 1, 2]:
 Pass
```

Class and object.

Python is an object-oriented programming language that offers classes, which are a potent tool for writing reusable code. To describe objects with shared characteristics and behaviours, classes are utilised.

Classes in Python:

In Python, a class is a user-defined data type that contains both the data itself and the methods that may be used to manipulate it. In a sense, classes serve as a template to create objects. They provide the characteristics and operations that the objects will employ.

Creating Classes in Python

In Python, a class can be created by using the keyword class, followed by the class name. The syntax to create a class is given below.

Syntax

```
Class ClassName:
   #statement_suite
```

In Python, we must notice that each class is associated with a documentation string which can be accessed by using <class-name>.__doc__. A class contains a statement suite including fields, constructor, function, etc. definition.

Objects in Python:

An object is a particular instance of a class with unique characteristics and functions. After a class has been established, you may make objects based on it. By using the class constructor, you may create an object of a class in Python. The object's attributes are initialised in the constructor, which is a special procedure with the name __init__.

Syntax:

```
# Declare an object of a class
Object_name = Class_Name(arguments)
```

The __init__() Function

To understand the meaning of classes we have to understand the built-in __init__() function.

All classes have a function called __init__(), which is always executed when the class is being initiated.

Use the __init__() function to assign values to object properties, or other operations that are necessary to do when the object is being created.

The __str__() Function

The __str__() function controls what should be returned when the class object is represented as a string.

If the __str__() function is not set, the string representation of the object is returned:

The self Parameter

The self parameter is a reference to the current instance of the class, and is used to access variables that belongs to the class.

It does not have to be named self , you can call it whatever you like, but it has to be the first parameter of any function in the class.

Function and function argument.

Python Functions is a block of statements that return the specific task. The idea is to put some commonly or repeatedly done tasks together and make a function so that instead of writing the same code again and again for different inputs, we can do the function calls to reuse code contained in it over and over again.

Some Benefits of Using Functions

Increase Code Readability

Increase Code Reusability

Python Function Declaration

The syntax to declare a function is:

Def function _name(parameters):

#statement

Return Expression

Types of Functions in Python

There are mainly two types of functions in Python.

Built-in library function: These are Standard functions in Python that are available to use.

User-defined function: We can create our own functions based on our requirements

Creating a Function in Python

We can create a user-defined function in Python, using the def keyword. We can add any type of functionalities and properties to it as we require.

A simple Python function

Def fun():

  Print("Welcome to GFG")

Calling a Python Function

After creating a function in Python we can call it by using the name of the function followed by parenthesis containing parameters of that particular function.

```
# A simple Python function

Def fun():

   Print("Welcome to GFG")

# Driver code to call a function

Fun()
```

Output:

Welcome to GFG

Python Function Arguments

Arguments are the values passed inside the parenthesis of the function. A function can have any number of arguments separated by a comma.

Types of Python Function Arguments

Python supports various types of arguments that can be passed at the time of the function call. In Python, we have the following 4 types of function arguments.

Default Arguments

A default argument is a parameter that assumes a default value if a value is not provided in the function call for that argument. The following example illustrates Default arguments.

Like C++ default arguments, any number of arguments in a function can have a default value. But once we have a default argument, all the arguments to its right must also have default values.

Keyword Arguments

The idea is to allow the caller to specify the argument name with values so that the caller does not need to remember the order of parameters.

Positional Arguments

We used the Position argument during the function call so that the first argument (or value) is assigned to name and the second argument (or value) is assigned to age. By changing the position, or if you forget the order of the positions, the values can be used in the wrong places

Arbitrary Keyword Arguments

In Python Arbitrary Keyword Arguments, *args, and **kwargs can pass a variable number of arguments to a function using special symbols. There are two special symbols:

*args in Python (Non-Keyword Arguments)

**kwargs in Python (Keyword Arguments)

Inheritance in python.

One of the core concepts in object-oriented programming (OOP) languages is inheritance. It is a mechanism that allows you to create a hierarchy of classes that share a set of properties and methods by deriving a class from another class. Inheritance is the capability of one class to derive or inherit the properties from another class

Python Inheritance Syntax

The syntax of simple inheritance in Python is as follows:

```
Class BaseClass:
   {Body}
Class DerivedClass(BaseClass):
   {Body}
```

Single Inheritance:

Single inheritance enables a derived class to inherit properties from a single parent class, thus enabling code reusability and the addition of new features to existing code.

Multilevel inheritance:

Multi-Level inheritance is possible in python like other object-oriented languages. Multi-level inheritance is archived when a derived class inherits another derived class. There is no limit on the number of levels up to which, the multi-level inheritance is archived in python.

Syntax

```
Class class1:
   <class-suite>
Class class2(class1):
   <class suite>
Class class3(class2):
   <class suite>
.
.
```

Python Multiple inheritance

When a class can be derived from more than one base class this type of inheritance is called multiple inheritances. In multiple inheritances, all the features of the base classes are inherited into the derived class.

Python provides us the flexibility to inherit multiple base classes in the child class.

The syntax to perform multiple inheritance is given below.

Syntax

```
Class Base1:
   <class-suite>
Class Base2:
   <class-suite>
.
.
.
Class BaseN:
   <class-suite>
Class Derived(Base1, Base2, …… BaseN):
   <class-suite>
```

Hierarchical Inheritance:

When more than one derived class are created from a single base this type of inheritance is called hierarchical inheritance. In this program, we have a parent (base) class and two child (derived) classes.

Hybrid Inheritance:

Inheritance consisting of multiple types of inheritance is called hybrid inheritance.

Polymorphism in Python

What is Polymorphism: The word polymorphism means having many forms. In programming, polymorphism means the same function name (but different signatures) being used for different types. The key difference is the data types and number of arguments used in function.

Polymorphism in Python using inheritance and method overriding:

```
Class Animal:
   Def speak(self):
      Raise NotImplementedError("Subclass must implement this method")
Class Dog(Animal):
   Def speak(self):
      Return "Woof!"
Class Cat(Animal):
   Def speak(self):
      Return "Meow!"
# Create a list of Animal objects
Animals = [Dog(), Cat()]
# Call the speak method on each object
For animal in animals:
Print(animal.speak())
```

Output

Woof!

Meow!

Function Polymorphism

An example of a Python function that can be used on different objects is the len() function.

String

For strings len() returns the number of characters:

```
X = "Hello World!"
Print(len(x))
```

Standard Modules.

A Python module is a file containing Python definitions and statements. A module can define functions, classes, and variables. A module can also include runnable code. Grouping related code into a module makes the code easier to understand and use. It also makes the code logically organized.

Create a Python Module

Let's create a simple calc.py in which we define two functions, one add and another subtract.

```
Def add(x, y):
   Return (x+y)
Def subtract(x, y):
   Return (x-y)
```

Import module in Python

We can import the functions, and classes defined in a module to another module using the import statement in some other Python source file.When the interpreter encounters an import statement, it imports the module if the module is present in the search path. A search path is a list of directories that the interpreter searches for importing a module. For example, to import the module calc.py, we need to put the following command at the top of the script.

Syntax of Python Import

Import module

Importing modules in Python Example

Now, we are importing the calc that we created earlier to perform add operation.

```
# importing  module calc.py
Import calc
Print(calc.add(10, 2))
```

Output:

12

Python Import From Module

Python's from statement lets you import specific attributes from a module without importing the module as a whole

Import all Names

The * symbol used with the import statement is used to import all the names from a module to a current namespace.

Syntax:

```
From module_name import *
```

Packages.

We usually organize our files in different folders and subfolders based on some criteria, so that they can be managed easily and efficiently. For example, we keep all our games in a Games folder and we can even subcategorize according to the genre of the game or something like this. The same analogy is followed by the packages in Python.

Python modules may contain several classes, functions, variables, etc. whereas Python packages contain several modules. In simpler terms, Package in Python is a folder that contains various modules as files.

Creating Package

Let's create a package in Python named mypckg that will contain two modules mod1 and mod2. To create this module follow the below steps:

Create a folder named mypckg.

Inside this folder create an empty Python file i.e. __init__.py

Then create two modules mod1 and mod2 in this folder.

Mod1.py

```
Def gfg():
   Print("Welcome to GFG")
```

Mod2.py

```
Def sum(a, b):
   Return a+b
```

Understanding __init__.py

__init__.py helps the Python interpreter recognize the folder as a package. It also specifies the resources to be imported from the modules. If the __init__.py is empty this means that all the functions of the modules will be imported. We can also specify the functions from each module to be made available.

__init__.py

```
From .mod1 import gfg

From .mod2 import sum
```

This __init__.py will only allow the gfg and sum functions from the mod1 and mod2 modules to be imported.

Import Modules from a Package

We can import these Python modules using the from…import statement and the dot(.) operator.

Syntax:

```
Import package_name.module_name
```

Exception raising.

The raise statement in Python is used to raise an exception. Try-except blocks can be used to manage exceptions, which are errors that happen while a programme is running. When an exception is triggered, the programme goes to the closest exception handler, interrupting the regular flow of execution.

The raise keyword is typically used inside a function or method, and is used to indicate an error condition.

We can throw an exception and immediately halt the running of your programme by using the raise keyword.

Python looks for the closest exception handler, which is often defined using a try-except block, when an exception is triggered.

If an exception handler is discovered, its code is performed, and the try-except block's starting point is reached again.

If an exception handler cannot be located, the software crashes and an error message appears.

```
X = -1


If x < 0:

  Raise Exception("Sorry, no numbers below zero")
```

You can raise any type of exception, such as ValueError, TypeError, KeyError, etc. to indicate specific errors in your code.

```
Def divide(a, b):

   If b == 0:

      Raise Exception(" Cannot divide by zero. ")

   Return a / b

Try:

   Result = divide(10, 0)

Except Exception as e:

   Print(" An error occurred: ", e)
```

Output:

An error occurred: Age must be 18 or above.

Exception handling

Different types of exceptions in python:

In Python, there are several built-in Python exceptions that can be raised when an error occurs during the execution of a program. Here are some of the most common types of exceptions in Python:

SyntaxError: This exception is raised when the interpreter encounters a syntax error in the code, such as a misspelled keyword, a missing colon, or an unbalanced parenthesis.

TypeError: This exception is raised when an operation or function is applied to an object of the wrong type, such as adding a string to an integer.

NameError: This exception is raised when a variable or function name is not found in the current scope.

IndexError: This exception is raised when an index is out of range for a list, tuple, or other sequence types.

KeyError: This exception is raised when a key is not found in a dictionary.

ValueError: This exception is raised when a function or method is called with an invalid argument or input, such as trying to convert a string to an integer when the string does not represent a valid integer.

AttributeError: This exception is raised when an attribute or method is not found on an object, such as trying to access a non-existent attribute of a class instance.

IOError: This exception is raised when an I/O operation, such as reading or writing a file, fails due to an input/output error.

ZeroDivisionError: This exception is raised when an attempt is made to divide a number by zero.

ImportError: This exception is raised when an import statement fails to find or load a module.

Try and Except Statement – Catching Exceptions

Try and except statements are used to catch and handle exceptions in Python. Statements that can raise exceptions are kept inside the try clause and the statements that handle the exception are written inside except clause.

Error Processing.

Errors are the problems in a program due to which the program will stop the execution. On the other hand, exceptions are raised when some internal events occur which changes the normal flow of the program.

Two types of Error occurs in python.

Syntax errors

Logical errors (Exceptions)

Syntax errors

When the proper syntax of the language is not followed then a syntax error is thrown.

Amount = 10000

check that You are eligible to

purchase Dsa Self Paced or not

If(amount>2999)

  Print("You are eligible to purchase Dsa Self Paced")

Output:

```
  File "/home/ac35380186f4ca7978956ff46697139b.py", line 4
    if(amount>2999)
                  ^
SyntaxError: invalid syntax
```

It returns a syntax error message because after the if statement a colon: is missing. We can fix this by writing the correct syntax.

Logical errors(Exception)

When in the runtime an error that occurs after passing the syntax test is called exception or logical type. For example, when we divide any number by zero then the ZeroDivisionError exception is raised, or when we import a module that does not exist then ImportError is raised.

Marks = 1000

perform division with 0

A = marks / 0

Print(a)

Output:

```
Traceback (most recent call last):
  File "/home/f3ad05420ab851d4bd106ffb04229907.py", line 4, in <module>
    a=marks/0
ZeroDivisionError: division by zero
```

Getting MySQL for python

Python MySQL Connector is a Python driver that helps to integrate Python and MySQL. This Python MySQL library allows the conversion between Python and MySQL data types. MySQL Connector API is implemented using pure Python and does not require any third-party library.

Installation

To install the Python-mysql-connector module, one must have Python and PIP, preinstalled on their system. If Python and pip are already installed type the below command in the terminal.

Pip3 install mysql-connector-python

Creating Database

After connecting to the MySQL server let's see how to create a MySQL database using Python. For this, we will first create a cursor() object and will then pass the SQL command as a string to the execute() method. The SQL command to create a database is –

CREATE DATABASE DATABASE_NAME

Creating Tables

For creating tables we will follow the similar approach of writing the SQL commands as strings and then passing it to the execute() method of the cursor object. SQL command for creating a table is –

```
CREATE TABLE
(
   Column_name_1 column_Data_type,
   Column_name_2 column_Data_type,
   :
   :
   Column_name_n column_Data_type
);
```

Insert Data into Tables

To insert data into the MySQL table Insert into query is used.

Syntax:

INSERT INTO table_name (column_names) VALUES (

Fetching Data

We can use the select query on the MySQL tables in the following ways –

In order to select particular attribute columns from a table, we write the attribute names.

SELECT attr1, attr2 FROM table_name

Update Data

The update query is used to change the existing values in a database. By using update a specific value can be corrected or updated. It only affects the data and not the structure of the table. The basic advantage provided by this command is that it keeps the table accurate.

Syntax:

UPDATE tablename

SET =”new value”

WHERE =”old value”;

Delete Data from Table

We can use the Delete query to delete data from the table in MySQL.

Syntax:

DELETE FROM TABLE_NAME WHERE ATTRIBUTE_NAME = ATTRIBUTE_VALUE

Drop Tables

Drop command affects the structure of the table and not data. It is used to delete an already existing table. For cases where you are not sure if the table to be dropped exists or not DROP TABLE IF EXISTS command is used. Both cases will be dealt with in the following examples.

Syntax:

DROP TABLE tablename;

DROP TABLE IF EXISTS tablename;

Where Clause

Where clause is used in MySQL database to filter the data as per the condition required. You can fetch, delete or update a particular set of data in MySQL database by using where clause.

Syntax:

SELECT column1, column2, …. columnN FROM [TABLE NAME] WHERE [CONDITION];

Connecting with database.

Python is a high-level, general-purpose, and very popular programming language. Basically, it was designed with an emphasis on code readability, and programmers can express their concepts in fewer lines of code. We can also use Python with SQL. In this article, we will learn how to connect SQL with Python using the 'MySQL Connector Python module. The diagram given below illustrates how a connection request is sent to MySQL connector Python, how it gets accepted from the database and how the cursor is executed with result data.

Connecting MySQL with Python

To create a connection between the MySQL database and Python, the connect() method of mysql.connector module is used. We pass the database details like HostName, username, and the password in the method call, and then the method returns the connection object.

The following steps are required to connect SQL with Python:

Step 1: Download and Install the free MySQL database from Online.

Step 2: After installing the MySQL database, open your Command prompt.

Step 3: Navigate your Command prompt to the location of PIP.

Step 4: Now run the commands given below to download and install "MySQL Connector". Here, mysql.connector statement will help you to communicate with the MySQL database.

Download and install "MySQL Connector"

Pip install mysql-connector-python

Step 5: Test MySQL Connector

To check if the installation was successful, or if you already installed "MySQL Connector", go to your IDE and run the given below code :

Import mysql.connector

Step 6: Create Connection

Now to connect SQL with Python, run the code given below in your IDE.

Passing query to MySQL.

Python GUI – tkinter

Python offers multiple options for developing GUI (Graphical User Interface). Out of all the GUI methods, tkinter is the most commonly used method. It is a standard Python interface to the Tk GUI toolkit shipped with Python. Python tkinter is the fastest and easiest way to create GUI applications. Creating a GUI using tkinter is an easy task.

To create a tkinter Python app:

Importing the module – tkinter

Create the main window (container)

Add any number of widgets to the main window

Apply the event Trigger on the widgets.

Importing a tkinter is the same as importing any other module in the Python code. Note that the name of the module in Python 2.x is 'Tkinter' and in Python 3.x it is 'tkinter'.

Import tkinter

There are two main methods used which the user needs to remember while creating the Python application with GUI.

Tk(screenName=None, baseName=None, className='Tk', useTk=1): To create a main window, tkinter offers a method 'Tk(screenName=None, baseName=None, className='Tk', useTk=1)'. To change the name of the window, you can change the className to the desired one. The basic code used to create the main window of the application is:

M=tkinter.Tk() where m is the name of the main window object

Mainloop(): There is a method known by the name mainloop() is used when your application is ready to run. Mainloop() is an infinite loop used to run the application, wait for an event to occur and process the event as long as the window is not closed.

m.mainloop()

Creating labels

Label: It refers to the display box where you can put any text or image which can be updated any time as per the code. The general syntax is:

W=Label(master, option=value)

Master is the parameter used to represent the parent window.

There are number of options which are used to change the format of the widget. Number of options can be passed as parameters separated by commas. Some of them are listed below.

Bg: to set the normal background color.

Bg to set the normal background color.

Command: to call a function.

Font: to set the font on the button label.

Image: to set the image on the button.

Width: to set the width of the button.

Height" to set the height of the button.

From tkinter import *

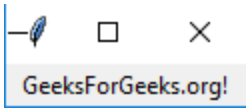
Root = Tk()

W = Label(root, text='GeeksForGeeks.org!')

w.pack()

root.mainloop()

Output:



Radio button

: It is used to offer multi-choice option to the user. It offers several options to the user and the user has to choose one option. The general syntax is:

W = RadioButton(master, option=value)

There are number of options which are used to change the format of this widget. Number of options can be passed as parameters separated by commas. Some of them are listed below.

Activebackground: to set the background color when widget is under the cursor.

Activeforeground: to set the foreground color when widget is under the cursor.

Bg: to set the normal background color.

Command: to call a function.

Font: to set the font on the button label.

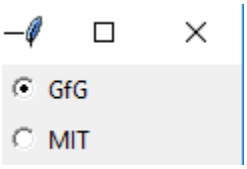
Image: to set the image on the widget.

Width: to set the width of the label in characters.

Height: to set the height of the label in characters.

```
From tkinter import *

Root = Tk()

V = IntVar()

Radiobutton(root, text='GfG', variable=v, value=1).pack(anchor=W)

Radiobutton(root, text='MIT', variable=v, value=2).pack(anchor=W)

Mainloop()
```

Output:



Text:

Text: To edit a multi-line text and format the way it has to be displayed. The general syntax is:

```
W  =Text(master, option=value)
```

There are number of options which are used to change the format of the text. Number of options can be passed as parameters separated by commas. Some of them are listed below.

Highlightcolor: To set the color of the focus highlight when widget has to be focused.

Insertbackground: To set the background of the widget.

Bg: to set the normal background color.

Font: to set the font on the button label.

Image: to set the image on the widget.

Width: to set the width of the widget.

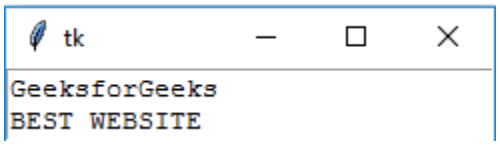
Height: to set the height of the widget.

```
From tkinter import *

Root = Tk()

T = Text(root, height=2, width=30)

T.pack()
```

T.insert(END, ‘GeeksforGeeks\nBEST WEBSITE\n’)

Mainloop()

Output:



Check button:

CheckButton: To select any number of options by displaying a number of options to a user as toggle buttons. The general syntax is:

W = CheckButton(master, option=value)

There are number of options which are used to change the format of this widget. Number of options can be passed as parameters separated by commas. Some of them are listed below.

Title: To set the title of the widget.

Activebackground: to set the background color when widget is under the cursor.

Activeforeground: to set the foreground color when widget is under the cursor.

Bg: to set the normal background color.

Command: to call a function.

Font: to set the font on the button label.

Image: to set the image on the widget.

From tkinter import *

Master = Tk()

Var1 = IntVar()

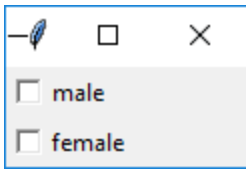
Checkbutton(master, text=’male’, variable=var1).grid(row=0, sticky=W)

Var2 = IntVar()

Checkbutton(master, text=’female’, variable=var2).grid(row=1, sticky=W)

Mainloop()

Output:



Button:

Button:To add a button in your application, this widget is used. The general syntax is:

W=Button(master, option=value)

Master is the parameter used to represent the parent window. There are number of options which are used to change the format of the Buttons. Number of options can be passed as parameters separated by commas. Some of them are listed below.

Activebackground: to set the background color when button is under the cursor.

Activeforeground: to set the foreground color when button is under the cursor.

Bg: to set the normal background color.

Command: to call a function.

Font: to set the font on the button label.

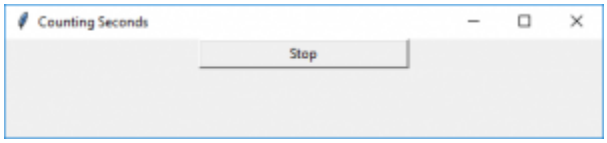
Image: to set the image on the button.

Width: to set the width of the button.

Height: to set the height of the button.

```
Import tkinter as tk

R = tk.Tk()

r.title('Counting Seconds')

button = tk.Button(r, text='Stop', width=25, command=r.destroy)

button.pack()

r.mainloop()
```

Output:



Listbox: It offers a list to the user from which the user can accept any number of options. The general syntax is:

W = Listbox(master, option=value)

Master is the parameter used to represent the parent window.

Listbox:

There are number of options which are used to change the format of the widget. Number of options can be passed as parameters separated by commas. Some of them are listed below.

Highlightcolor: To set the color of the focus highlight when widget has to be focused.

Bg: to set the normal background color.

Bd: to set the border width in pixels.

Font: to set the font on the button label.

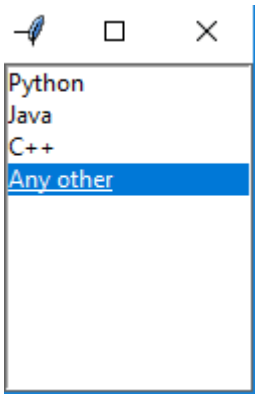
Image: to set the image on the widget.

Width: to set the width of the widget.

Height: to set the height of the widget.

```
From tkinter import *

Top = Tk()

Lb = Listbox(top)

Lb.insert(1, 'Python')

Lb.insert(2, 'Java')

Lb.insert(3, 'C++')

Lb.insert(4, 'Any other')

Lb.pack()

Top.mainloop()
```

Output:



Connecting with database

Connecting to the Database

The mysql.connector provides the connect() method used to create a connection between the MySQL database and the Python application. The syntax is given below.

Syntax:

```
Conn_obj= mysql.connector.connect(host = <hostname>, user = <username>, passwd = <password>)
```

The connect() function accepts the following arguments.

Hostname – It represents the server name or IP address on which MySQL is running.

Username – It represents the name of the user that we use to work with the MySQL server. By default, the username for the MySQL database is root.

Password – The password is provided at the time of installing the MySQL database. We don't need to pass a password if we are using the root.

Database – It specifies the database name which we want to connect. This argument is used when we have multiple databases.

Django installation

To install Django, first visit to django official site (https://www.djangoproject.com) and download django by clicking on the download section. Here, we will see various options to download The Django.

Django requires pip to start installation. Pip is a package manager system which is used to install and manage packages written in python. For Python 3.4 and higher versions pip3 is used to manage packages.

we are installing Django in Ubuntu operating system

The complete installation process is described below. Before installing make sure pip is installed in local system.

Here, we are installing Django using pip3, the installation command is given below.

$ pip3 install django==2.0.3

Verify Django Installation

After installing Django, we need to varify the installation. Open terminal and write python3 and press enter. It will display python shell where we can verify django installation.

Look at the Django version displayed by the print method of the python. Well, Django is installed successfully. Now, we can build Django web applications.

Create a Project in Django

Before you begin, you will need to have the following:

Windows 10 or later

Python 3.6 or later

Python pip

Step 1: Opening PowerShell

The first step is to open PowerShell. To do this, press the Windows key and type PowerShell. Right-click on the PowerShell icon and select Run as Administrator.

Step 2: Verifying Python Installation

Once PowerShell is open, you can verify that Python is installed by typing python –version. This will display the version of Python that is installed.

Terminal command:

Python –version

Step 3: Upgrading PIP

Next, you will need to upgrade pip. To do this, type python -m pip install –upgrade pip. This will ensure that you have the latest version of pip installed.

Step 4: Creating a Project Directory

Now, you will need to create a project directory. To do this, type mkdir myproject. This will create a directory called myproject.

(i) Create the directory by utilizing the mkdir command. This command is a powerful tool that allows you to quickly and easily create a directory in your system. With this command, you can create a directory in seconds, making it a great time-saver for any user.

Mkdir django_project

(ii) Navigate to the Django_Project directory using the `cd` command. Here, you'll find all the necessary components to get your project up and running.

Cd django_project

Step 5: Creating the Virtual Environment

Next, you will need to create a virtual environment. To create a virtual environment called myproject.

To do this, type in terminal

Python -m venv myproject

Step 6: Activating the Virtual Environment

Now, you will need to activate the virtual environment. To do this, type

Venv\Scripts\activate

Step 7: Installing Django

This section will teach you how to install Django on your system using pip. To do this, run the following command:

Pip install Django

This will install the latest version of Django, allowing you to start building powerful web applications easily.

Step 8: Create the Django Project

Now it's time to create a project. According to the Django documentation, a project is a Python package – i.e. a directory of code – that contains all the settings for an instance of Django. This includes database configuration, Django-specific options, and application-specific settings.

To create the project, use the command-line utility django-admin that comes with Django. This command will generate files where you can configure the settings for your database, add third-party packages, and more.

Create the project using the following command:

Django-admin startproject test_project

Change into the test_project directory:

Cd test_project

Type the following command to see the contents in the project directory:

Ls test_project

The directory test_project contains Django configuration files. The manage.py file is especially useful when starting a development server, which is what you will do in the next step.

Step 9: Running the Development Server

Now that the project has been created, it's time to start the Django development server.Start the development server using the manage.py runserver command:

Note: Make sure sure you are in the same directory as manage.py file after to run manage.py runserver

Python manage.py runserver

Step 10: Create app in django

Now as we created the project in django and our django server is up and running and if now to create app in django project we can achieve it b by using django inbuit command which will initiate the django app for us.

Note: Make sure before running the command we must inside our project directory if not then run the below command

Cd test_project

Now we are inside the test_project and we are ready to create django app

Django-admin startapp test_app

Method-1

To create a basic app in your Django project you need to go to the directory containing manage.py and from there enter the command :

Python manage.py startapp projectApp

Method-2

To create a basic app in your Django project you need to go to the directory containing manage.py and from there enter the command :

Django-admin startapp projectApp

Template and model

Templates are the third and most important part of Django's MVT Structure. A template in Django is basically written in HTML, CSS, and Javascript in a .html file. Django framework efficiently handles and generates dynamic HTML web pages that are visible to the end-user. Django mainly functions with a backend so, in order to provide a frontend and provide a layout to our website, we use templates. There are two methods of adding the template to our website depending on our needs.

We can use a single template directory which will be spread over the entire project. For each app of our project, we can create a different template directory.

Templates in Django

For our current project, we will create a single template directory that will be spread over the entire project for simplicity. App-level templates are generally used in big projects or in case we want to provide a different layout to each component of our webpage.

Configuration

Settings.py: Django Templates can be configured in app_name/settings.py,

```
TEMPLATES = [
  {
    # Template backend to be used, For example Jinja
    'BACKEND': 'django.template.backends.django.DjangoTemplates',
    # Directories for templates
    'DIRS': [],
    'APP_DIRS': True,
    # options to configure
    'OPTIONS': {
      'context_processors': [
        'django.template.context_processors.debug',
        'django.template.context_processors.request',
        'django.contrib.auth.context_processors.auth',
        'django.contrib.messages.context_processors.messages',
      ],
    },

  },
]
```

Data manipulation

Django's Object-Relational Mapping (ORM) system, a fundamental component that bridges the gap between the database and the application's code. This article delves into the intricate realm of Python, Django, and their ORM, uncovering how this technology stack simplifies database interactions and accelerates the development process.

Insert Data with Django ORM

To create an object of model Album and save it into the database, we need to write the following command:

```
>>> a = Album(title = "Divide", artist = "Ed Sheeran", genre = "Pop")

>>> a.save()
```

To create an object of model Song and save it into the database, we need to write the following command:

```
>>> s = Song(name = "Castle on the Hill", album = a)

>>> s.save()
```

Creating Data with Django ORM

Let us add 2 more Albums records for the sake of demonstration.

```
>>> a = Album(title = "Abbey Road", artist = "The Beatles", genre = "Rock")

>>> a.save()

>>> a = Album(title = "Revolver", artist = "The Beatles", genre = "Rock")

>>> a.save()
```

Retrieving Data with Django ORM

To retrieve all the objects of a model, we write the following command:

```
>>> Album.objects.all()

<QuerySet [<Album: Divide>, <Album: Abbey Road>,
```

Update Data with Django ORM

We can modify an existing object as follows:

```
>>> a = Album.objects.get(pk = 3)

>>> a.genre = "Pop"

>>> a.save()
```

Deleting Data with Django ORM

To delete a single object, we need to write the following commands:

```
>>> a = Album.objects.get(pk = 2)

>>> a.delete()

>>> Album.objects.all()

<QuerySet [<Album: Divide>, <Album: Revolver>]>
```

Django Admin

Django Admin is a really great tool in Django, it is actually a CRUD* user interface of all your models!

*CRUD stands for Create Read Update Delete.

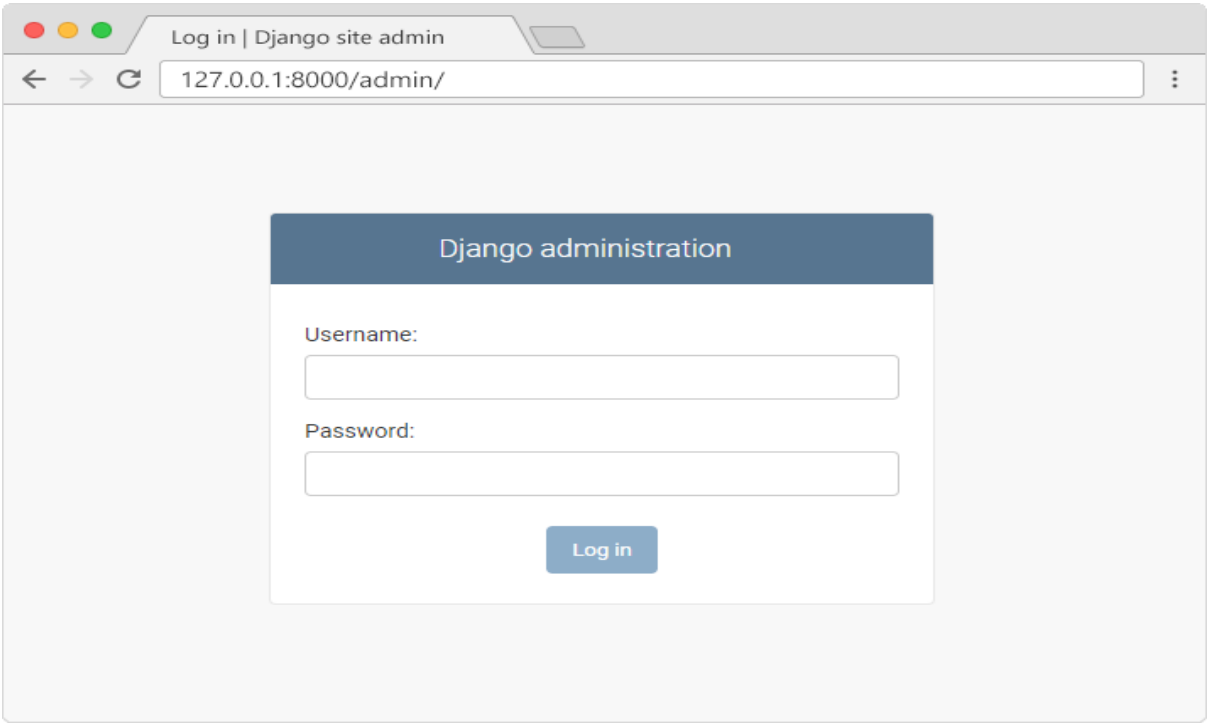
Getting Started

To enter the admin user interface, start the server by navigating to the /myworld folder and execute this command:

Py manage.py runserver

In the browser window, type 127.0.0.1:8000/admin/ in the address bar.

The result should look like this:



Django Template Variables

Variables – Django Templates

A Django template is a text document or a Python string marked-up using the Django template language. Django being a powerful Batteries included framework provides convenience to rendering data in a template. Django templates not only allow passing data from view to template, but also provides some limited features of a programming such as variables, for loops, etc.

This article revolves about how to use a variable in Template. A variable outputs a value from the context, which is a dict-like object mapping keys to values.

Syntax:

{{ variable_name }}

Example:

Variables are surrounded by {{ and }} like this:

My first name is {{ first_name }}. My last name is {{ last_name }}.

With a context of {'first_name': 'Naveen', 'last_name': 'Arora'}, this template renders to:

My first name is Naveen. My last name is Arora.

Create Variables in Template

You can also create variables directly in the template, by using the {% with %} template tag.

The variable is available until the {% endwith %} tag appears:

Example

Templates/template.html:

{% with firstname="Tobias" %}

<h1>Hello {{ firstname }}, how are you?</h1>

{% endwith %}

Django Template Tags

Django Web Framework ships with dozens of tags used to implement arbitrary logics right in the template. Tags look like this: {% tag %}. Tags are more complex than variables: Some create text in the output, some control flow by performing loops or logic, and some load external information into the template to be used by later variables. Tags provide arbitrary logic in the rendering process. For example, a tag can output content, serve as a control structure e.g. an "if" statement or a "for" loop, grab content from a database, or even enable access to other template tags.

Syntax

{% tag_name %}

Example

Tags are surrounded by {% and %} like this:

{% csrf_token %}

Most tags accept arguments, for example :

{% cycle ‘odd’ ‘even’ %}

If tag

The {% if %} tag evaluates a variable, and if that variable is “true” (i.e. exists, is not empty, and is not a false boolean value) the contents of the block are output.Example

```
{% if athlete_list %}

   Number of athletes: {{ athlete_list|length }}

{% elif athlete_in_locker_room_list %}

   Athletes should be out of the locker room soon!

{% else %}

   No athletes.

{% endif %}
```

In the above, if athlete_list is not empty, the number of athletes will be displayed by the {{ athlete_list|length }} variable.

As one can see, the if tag may take one or several {% elif %} clauses, as well as an {% else %} clause that will be displayed if all previous conditions fail. These clauses are optional.

For loop

For tag loops over each item in an array, making the item available in a context variable.

Example

For example, to display a list of athletes provided in athlete_list:

```
<ul>

{% for athlete in athlete_list %}

   <li>{{ athlete.name }}</li>

{% endfor %}

</ul>
```

For … empty loop

For tag loops over each item in an array, making the item available in a context variable. The for tag can take an optional {% empty %} clause whose text is displayed if the given array is empty or could not be found. This is basically used as a condition to be followed to check if queryset is empty and what action to be performed in the same scenario.

Example

```
<ul>
{% if athlete_list %}
   {% for athlete in athlete_list %}
   <li>{{ athlete.name }}</li>
   {% endfor %}
{% else %}
   <li>Sorry, no athletes in this list.</li>
{% endif %}
</ul>
```

Database connection with MySQL in django

Django is a Python-based web framework that allows you to quickly create efficient web applications. It is also called batteries included framework because Django provides built-in features for everything including Django Admin Interface, default database – SQLlite3, etc.

Steps to connect MySQL to Django

Step 1: Create a new project and start the process to connect django with mysql.

Django-admin startproject MyDB

Step 2: Move to the MyDB folder.

Cd MyDB

Note: You should have MySQL installed and be able to log in with appropriate privileges to create a database.

Step 3: Once you're logged into MySQL, you can create a new database. Replace your_database_name with the desired name for your database.

CREATE DATABASE your_database_name

Step 4: Update the settings.py

Open settings.py here inside the DATABASES variable configure MySQL database values, and add values of your database.

```
DATABASES = {
   'default': {
      'ENGINE': 'django.db.backends.mysql',
```

```
        'NAME': 'mydb',
        'USER': 'root',
        'PASSWORD': 'admin',
        'HOST':'localhost',
        'PORT':'3306',
    }
}
```

NAME: It indicates the name of the database we want to connect.

USER: The MYSQL username is the one who has access to the database and manages it.

PASSWORD: It is the password of the database.

HOST: It is indicated by "127.0.0.1" and "PORT" "3306" that the MySQL database is accessible at hostname "0.0.1" and on port "3306."

Step 5: Run the server.

Python manage.py runserver

Step 6: Run the migration command

Python manage.py makemigrations

How to install python installation

Python installation : version 3.6.2

The python download requires about 30 mb of disk space; keep it on your

Machine,in case you need to re-install python.when installed ,python requires

About an additional 90 mb of disk space.

Step 1:-

Downloding:-

1.spiecify the given URL in web browser www.python.org/downlods.

Step 2:-

Click the download python 3.6.2 button the file named python-3.6.2.exe

Should be start downloading into your standard download folder. This file

About 30 mb might take a while to download fully if you are no a slow

Internet connection (it took me about 10 seconds over a cable modem.)

The file should appear as

Python-3.6.2.exe.

*Instaling:-

1.Double-click the icon labeling the file

Python-3.6.2.exe.

An open file –security warning pop-up window will appear.

2.Click run

A python 3.6.2(32-bit)setup pop-up windowwill appear.

Ensure that the install launcher for all uses (recommended) and the add

Python 3.6 to path check boxes at the bottom are checked.

-python installed on your computer;

Install Now….

3.Highlight the install now (or upgrade now) massage,and then click it.

-A user account control pop-up window with appear,posing the question ,do

You want the allow the following program to make changes to this computer .

4.Click the yes button .

-A now python 3.6.2(32bit)setup-pop-up window will appear with a setup

Progress messages and a progress bar.

-during installation ,it will show the various compents it is installing and

Move the progress bar towards completion ,so on ,a new python 3.6.2(32bit)

Set pop-up window will appear with a setup was successfully message.

5.Click the close button .

Python should now be installed.

*varifiying:-

To try,to verifiy installation,

1.navigate to the directonary

C:\users\pattis\AppData\local\program\python\python 3.6.2

Python as installed-installing step

2.Double-click the icon /file python.exe.

Python Installation Completed…..